

## 4 Network Models

It is often useful to study networks generated by following simple rules. These model networks can be analysed mathematically and compared to real world networks where they can suggest the mechanisms behind certain observations. We can also create random networks which are similar to our observed network in some way e.g. the same degree distribution, and compare our network to an ensemble of random ones to see if the property of interest is statistically significantly different than a random network.

### Reading

[1] **Chapters 3, 5**, [2] **Chapters 12-15**.

#### 4.1 Ensembles

We will discuss models in terms of the rules by which they generate particular networks but it can also be helpful to think of an **ensemble** (a very big set)  $\Omega$  of all the possible graphs which share some property. For example all the graphs with exactly  $N$  nodes and  $L$  edges. A model defines a probability distribution on  $\Omega$ ,  $P(G)$  which gives the probability of picking a particular graph from  $\Omega$  e.g.  $P(G) = 1/|\Omega|$  which would make each graph equally likely to be chosen. Given  $P$  and  $\Omega$  we can compute ensemble averages like

$$\langle d \rangle = \sum_{\Omega} P(G) d(G)$$

and

$$\langle d^2 \rangle = \sum_{\Omega} P(G) d^2(G)$$

Giving us the mean, variance ( $\sigma_d^2 = \langle d^2 \rangle - \langle d \rangle^2$ ) and other statistical measures of the property in the ensemble. We can then compare the value of the property computed on our network to this e.g. using a z-score  $z_d = \frac{d - \langle d \rangle}{\sigma_d}$  or p-value. Obviously this is quite hard to do analytically, but we can simulate a large number of networks, generating a distribution of possible values and use this to compute a p-value for the observed values of  $d$ .

## 4.2 Erdős Rényi Model

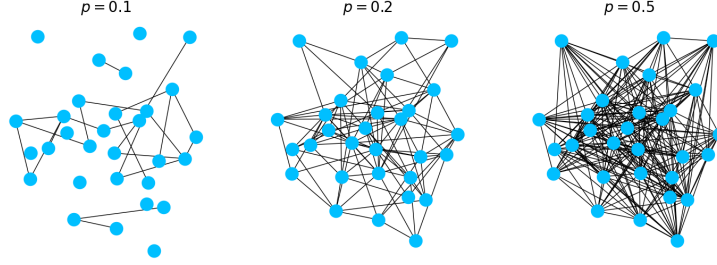


Figure 1: ER networks on 30 nodes with different edge probabilities.

The simplest model of a network is the Erdős Rényi (ER) or random graph model. There are some variants but the most common one is to start with  $N$  nodes and go through each of the possible  $\binom{N}{2} = \frac{N(N-1)}{2}$  edges and add each one with probability  $p$ .

Since each choice to add a link or not is made independently with the same probability this allows a fairly simple analysis of these random networks. The probability to have exactly  $L$  links is the probability of  $L$  successes (probability  $p^L$ ) and  $\binom{N}{2} - L$  failures (probability  $(1-p)^{\binom{N}{2}-L}$ ). The number of different networks of  $N$  nodes and  $L$  edges is  $\binom{\binom{N}{2}}{L}$  combining these gives

$$P(L) = \binom{\binom{N}{2}}{L} p^L (1-p)^{\binom{N}{2}-L} \quad (1)$$

This is a binomial probability distribution. Knowing this tells us the mean, variance and other statistics. They are not too hard to calculate but they are also well known so can be looked up. The mean number of edges for example is

$$\langle L \rangle = \binom{N}{2} p$$

i.e. just  $p$  times the number of potential edges. The average degree is

$$\langle k \rangle = \frac{2\langle L \rangle}{N} = (N-1)p$$

A very important observation about random graphs concerns their degree distributions. The probability that a node has  $k$  links is the product of

1. The probability for  $k$  successes  $p^k$
2. The probability of  $N - k$  failures  $(1 - p)^{N-k}$
3. The number of ways of choosing  $k$  links to  $N - 1$  other nodes

multiplying these gives,

$$p_k = \binom{N-1}{k} p^k (1-p)^{N-k}. \quad (2)$$

This is another binomial distribution. The average value of  $k$  is

$$\langle k \rangle = (N-1)p$$

as before. Since the binomial is discrete, it is harder to work with analytically. We make the approximation that  $N \gg \langle k \rangle$ , which is typically true in practice. A standard argument shows that in this limit we can very accurately approximate a binomial distribution with a Poisson distribution

$$p_k \simeq e^{-\langle k \rangle} \frac{\langle k \rangle^k}{k!} \quad (3)$$

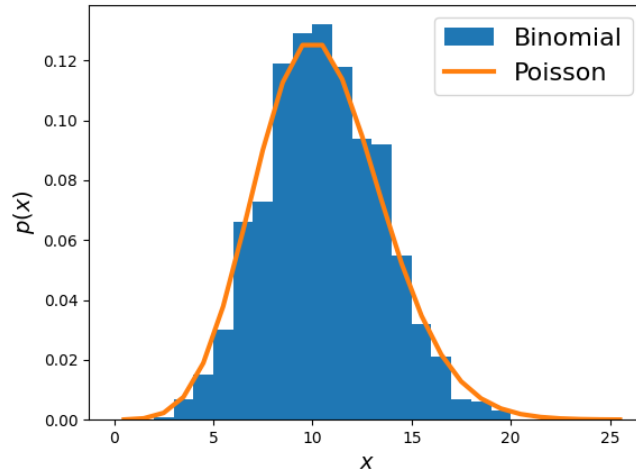


Figure 2: Binomial with  $N = 100$ ,  $p = 0.1$  compared to Poisson with  $\langle k \rangle = 10$ .

Figure 2 shows that 1. the Poisson approximation is very accurate and 2. the probability  $p_k$  decreases rapidly with  $k$ . This is because  $k!$  increases much faster than  $c^k$  once  $k > c$ . This is **not** the case with real networks, where  $p_k$  typically decays much more slowly as a function of  $k$ . A result about Poisson distributions which we will use often is

$$\langle k^2 \rangle = \langle k \rangle (\langle k \rangle + 1) \quad (4)$$

Another way that random networks differ from real ones is their clustering. Typically real networks are highly clustered e.g. your friends probably know each other. To calculate clustering in a random network we start with the observation that for a node of degree  $k$ , there are  $k(k-1)/2$  possible edges between neighbours and therefore  $p \frac{k(k-1)}{2}$  of them are present on average.

$$C_i = \frac{2L_i}{k_i(k_i-1)} = \frac{2pk_i(k_i-1)/2}{k_i(k_i-1)} = p = \frac{\langle k \rangle}{N}$$

This says that with fixed  $\langle k \rangle$ , the clustering of a node decreases with increasing  $N$  and the clustering is independent of degree. Neither of these properties are true for real networks.

There is one way however in which real networks and random networks are alike, they both have small average path lengths. That is, the shortest distance between any two nodes tends to be small. To see this, pick a target node and note there are  $\langle k \rangle^0 = 1$  nodes a distance 0 away (the node itself!). There are  $\langle k \rangle^1$  nodes a distance 1 away. One way to represent a random network which can be useful is via the matrix

$$P = \begin{pmatrix} 0 & p & \dots p \\ p & 0 & \dots p \\ \dots & & \\ p & p & \dots 0 \end{pmatrix}$$

An  $N \times N$  matrix where all the non-diagonal entries are equal to  $p$ . We can treat this like the adjacency matrix where the results represent probabilities. We get

$$k_i = \sum_j P_{ij} = (N-1)p$$

which is just the average degree from before. Recall that  $A^2$  counts paths of length 2. So the number of paths of length two attached to a node  $i$  is

$$\sum_j (A^2)_{ij}$$

$$P^2 = \begin{pmatrix} (N-1)p^2 & (N-2)p^2 & \dots & (N-2)p^2 \\ (N-2)p^2 & (N-1)p^2 & \dots & (N-2)p^2 \\ \dots & \dots & \dots & \dots \\ (N-2)p^2 & (N-2)p^2 & \dots & (N-1)p^2 \end{pmatrix}$$

Since  $N$  is large  $N \simeq N-1 \simeq N-2$  and the number of nodes a distance 2 away is  $N^2 p^2 \simeq \langle k \rangle^2$ . We can keep going, so the number of nodes less than or equal to  $d$  away from the start is approximately  $\langle k \rangle^d$ . This grows very fast and it won't take many steps to cover all  $N$  nodes. We can get an idea of the growth by asking for the value of  $d$  when all the nodes in the network are reached,

$$\langle k \rangle^{d_{max}} \simeq N \quad (5)$$

solving for  $d_{max}$  gives

$$d_{max} \simeq \frac{\log N}{\log \langle k \rangle} \quad (6)$$

Since  $\log N \ll N$  average path lengths are quite small, as observed in e.g. 6 degrees of separation.

### 4.3 Watts-Strogatz Model

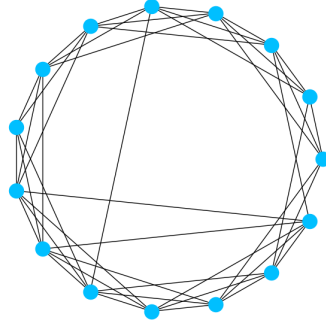


Figure 3: WS network with  $N = 15$ ,  $c = 6$ ,  $p = 0.1$ .

A famous model that shows how we can have high clustering and small path lengths is the **Watts-Strogatz** (WS) model. The basic idea is quite simple.

1. Take  $N$  nodes in a circle and connect each one to the  $c$  nearest neighbours.
2. With small probability  $p$  remove each link and replace it with a random one from the  $\binom{N}{2}$  possibilities.

An example is shown in Figure 3. For small values of  $p$  it is possible for these networks to have both high clustering and low path lengths, with the randomly added links acting as shortcuts. A variant of the model, where links are only added at random but not removed is also studied (called the Newman Watts Strogatz model). This is easier to study analytically but simply generating a large number on a computer is even easier!

#### 4.4 Barabási-Albert Model

The WS model still has a rather unrealistic degree distribution. Constructing networks with realistic, long tailed, degree distributions is the aim of the **Barabási-Albert** (BA) model. The key idea is growth by **preferential attachment**. Starting with  $m_0$  nodes where each one has at least 1 link, at each step, add a node with  $m$  links where the probability that the new node connects to the node  $i$  is

$$\frac{k_i}{\sum_j k_j} \quad (7)$$

This is just (half) the proportion of the total number of edges that are connected to  $i$ . In this way nodes which have more links are more attractive to new nodes. This is preferential attachment or *the rich get richer*.

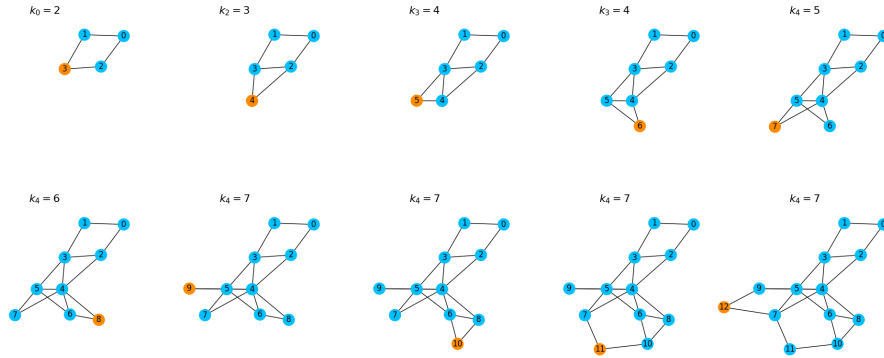


Figure 4: First steps of the BA network growth.

Figure 4 shows the first steps of the growth process along with the maximum degree at each step. At each step we add  $m$  nodes, so, at late times,  $t$  when we have added many more edges than we started with, the number of edges is approximately

$$L_t = mt$$

the number of new links added to node  $i$  on average at large times is

$$m \frac{k_i}{\sum_j^{t-1} k_i} = \frac{mk_i}{2m(t-1)} \simeq \frac{k_i}{2t}$$

We can approximate the discrete growth of the BA model with a continuous growth at this rate giving the differential equation

$$\frac{dk_i}{dt} = \frac{k_i}{2t}$$

Solving it gives

$$k_i(t) = m \left( \frac{t}{t_i} \right)^{1/2} \quad (8)$$

where  $t_i$  is the time that node  $i$  joins the network, where  $k_i(t_i) = m$ . The degree of each node increases with the square root of time. The rate of change of the node degree is

$$\frac{dk_i(t)}{dt} = \frac{m}{2\sqrt{t_i}} \frac{1}{\sqrt{t}} \quad (9)$$

Since  $t_i$  is smaller for older nodes,  $\frac{m}{2\sqrt{t_i}}$  is larger and older nodes acquire more links.

For some fixed number  $k$  we can compute the number of nodes with degree smaller than  $k$  from the above. If  $k_i < k$  then

$$m \left( \frac{t}{t_i} \right)^{1/2} < k$$

rearranging gives

$$t_i < \left( \frac{m}{k} \right)^2 t$$

that is, nodes added before the time on the right will have more than  $k$  links. Since we just add one node each timestep, the term on the right is the number of nodes with degree smaller than  $k$ . At time  $t$  there are  $t$  nodes

(neglecting the small number we started with) so the probability a node has degree larger than  $k$  is

$$P(k_i > k) = \frac{\left(\frac{m}{k}\right)^2 t}{t} = \left(\frac{m}{k}\right)^2 \quad (10)$$

And therefore the chance of having degree  $k_i$  or smaller is

$$P(k_i \leq k) = 1 - P(k_i > k) = 1 - \left(\frac{m}{k}\right)^2 \quad (11)$$

This is the cumulative distribution. Differentiating gives the corresponding probability distribution, which is the probability for a node to have degree  $k$ ,

$$\frac{dP(k_i \leq k)}{dk} = p(k) = 2m^2 k^{-3} \quad (12)$$

There are a couple of important things

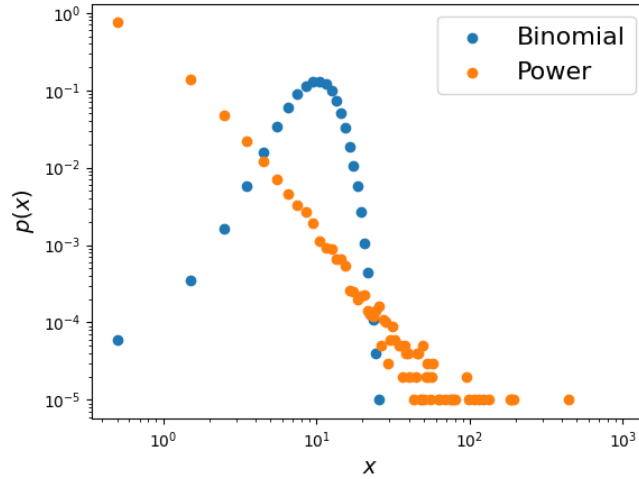


Figure 5: Binomial with  $N = 100$ ,  $p = 0.025$  and power law  $\sim k^{-3}$ ,  $10^5$  samples from each.

1. This decays with  $k$  but much more slowly than a Poisson distribution, as shown in Figure 5. Note the highest degree in the binomial (ER) model is around 10 while the highest degree in the BA model is around 500.

2. It is an example of a **power law**, in this case with **exponent 3**. This means the plot of  $p_k$  against  $k$  will be a straight line on logarithmic axes, with slope  $-3$ . This is the behaviour observed in many real networks, though with different exponents.

Because a preferential attachment model reproduces common features of real networks, this suggests something like this mechanism is at play in the real world.

#### 4.5 Configuration Model

Because the degree distribution is so important, a model where we specify the degree distribution and generate networks which realise that distribution is potentially very powerful. Many apparently unusual features of networks are actually inevitable consequences of their degree distributions. A model where we specify the degrees is called the **configuration model**.

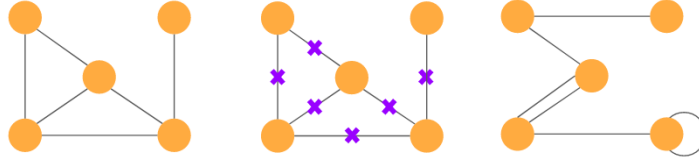


Figure 6: Configuration model on a small network.

Usually we start with an observed network and sever all the links, leaving the ‘stubs’ attached. We can also just make up a degree sequence  $k_i$  and imagine each node having  $k_i$  stubs coming out of it. If we don’t derive  $k_i$  from an existing network it is not guaranteed that we can construct a network with that degree sequence. A sequence where we can draw a network is called **graphical**. Generally, we just use degree sequences which come from existing networks and are therefore automatically graphical.

When we have a number of nodes and stubs, the configuration model joins the stubs randomly. The probability of a link between nodes  $i$  and  $j$  is

$$p_{ij} = \frac{k_i k_j}{2L - 1} \simeq \frac{k_i k_j}{2L} \quad (13)$$

That is, we have  $k_i$  stubs on node  $i$  which can be joined to any of the  $k_j$  stubs of node  $j$  and there are  $2L - 1$  total stubs it could have connected to. Since  $2L$  is usually large we, drop the  $-1$  for simplicity.



Figure 7: A swap move made during a degree preserving randomization.

A very important use of the configuration model is to produce **degree preserving randomizations**. Here the degree sequence comes from our observed network and we produce new networks with the same degree sequence. If some property on our network is explained by degree sequence alone, then that property will not be statistically significant in the ensemble of degree preserving randomizations of the original network. An algorithm to perform degree preserving randomization is to select two edges  $(a, b)$  and  $(c, d)$ , then swap them as in Figure 7, making new edges  $(a, d)$  and  $(c, b)$ . The degrees of the nodes are all preserved this way. We perform this process until every link has been rewired at least once.

One issue with this model is that there is nothing to stop the creation of self loops and multi-edges. Generally, for large graphs, these don't introduce a problem - the number of such edges is small. For example, the probability to have a 2-edge between  $i$  and  $j$  is

$$\sim \frac{k_i k_j (k_i - 1)(k_j - 1)}{(2L)^2}$$

thus the expected number of 2-edges is

$$N_2 = \frac{1}{2(2L)^2} \sum_{ij} k_i k_j (k_i - 1)(k_j - 1) = \frac{1}{2(2L)^2} \left( \sum_i k_i (k_i - 1) \right)^2$$

Using  $\langle k \rangle = \frac{2L}{N}$  we have

$$N_2 = \frac{1}{2} \frac{1}{\langle k \rangle^2 N^2} \left( \sum_i k_i^2 - N \langle k \rangle \right)^2 = \frac{1}{2} \left( \frac{\langle k^2 \rangle - \langle k \rangle^2}{\langle k \rangle} \right)^2$$

which is half the squared variance of  $k$ . Importantly, this does not depend on  $N$ , so the number of 2-edges stays constant as the network grows larger, thus multi-edges make up a smaller and smaller fraction of all edges.

A very similar model is the **Chung-Lu** (CL) model, in which the *expected* degrees are specified, rather than the exact ones. Rather than trying to

produce a graph with exactly the specified degree sequence the CL model computes  $p_{ij} = \frac{k_i k_j}{2L}$  and sets

$$A_{ij} = \begin{cases} 1 & \text{with probability } p_{ij} \\ 0 & \text{otherwise} \end{cases} \quad (14)$$

This generates networks whose degrees have expected values  $k_i$  (averaged over many random networks) though the individual networks will generally have different degree sequences which are different than  $k_i$ .

#### 4.6 Exponential Random Graphs

A very powerful and general model is the **exponential random graph** (ERG) model, which I will briefly touch on the basics of. We assume there is a ‘space’ or ensemble (i.e. a set) of all graphs with  $N$  nodes,  $\mathcal{G}$ . We imagine picking graphs from this set with some probability  $P(G)$  where we must have

$$\sum_{\mathcal{G}} P(G) = 1$$

If we knew  $P$  we could compute expectations like

$$\langle x \rangle = \sum_{\mathcal{G}} P(G) x(G)$$

The ERG model works backwards from this - we fix the expectations of certain graph properties (say the number of edges or average clustering) and use this to get determine  $P$ . If we have  $K$  properties  $x_i$  whose average we fix we have  $K$  constraints like

$$\sum_{\mathcal{G}} P(G) x_i(G) = \langle x_i \rangle$$

which we could solve to fix  $P$ . The number of constraints will generally be small (a handful at most) while the number of graphs with  $N$  vertices is  $2^{\binom{N}{2}}$ . Thus there are many functions  $P(G)$  which satisfy the constraints. To specify  $P$  fully we can attempt to satisfy the constraints while maximising the **entropy**

$$S = - \sum_{\mathcal{G}} P(G) \ln P(G) \quad (15)$$

To understand why maximising this function is a good way to find  $P$  would require a detour into information theory and statistical physics. Very simply, maximising it makes the least possible assumptions about the data.

To maximise a function with constraints we use the method of Lagrange multipliers

$$\mathcal{L} = - \sum_{\mathcal{G}} P(G) \ln P(G) + \alpha \left( \sum_{\mathcal{G}} -1 \right) + \sum_i^K \beta_i \left( \sum_{\mathcal{G}} P(G) x_i(G) - \langle x_i \rangle \right)$$

where  $\alpha$  and  $\beta_i$  are Lagrange multipliers. Differentiating with respect to  $P$ , equating to zero and solving gives

$$P(G) = \exp \left( \alpha - 1 + \sum_i \beta_i x_i(G) \right)$$

Which is usually rewritten as

$$P(G) = \frac{e^{H(G)}}{Z} \quad (16)$$

Copying names from statistical physics  $H$  is called the **graph Hamiltonian**

$$H(G) = \sum_i \beta_i x_i(G)$$

and  $Z$  is called the **partition function**, which is set by solving

$$\sum_{\mathcal{G}} P(G) = 1 \implies Z = \sum_{\mathcal{G}} e^{H(G)}$$

The values of  $\beta_i$  are found by substituting this back into the constraints and solving the  $K$  simultaneous equations. We won't go further with this, but there is a lot more development of this idea in the literature.

## References

- [1] Márton Pósfai and Albert-László Barabási. *Network science*. Cambridge University Press, 2016.
- [2] Mark Newman. *Networks*. Oxford university press, 2018.